

Lean for AI, AI for Lean

Leonardo de Moura

Lean FRO | March 2026

The Result

10 AI agents built a **verified embedded DSL** with proved optimizations in **a single weekend**.

- Zero **sorry** — 52 theorems, all complete
- 5 verified compiler optimizations
- Determinism, type safety, memory safety — all proved
- Linear ownership typing with soundness proof
- Interpreter correctness — sound and complete
- 27 modules, ~7,400 lines of Lean

Lessons Learned

- I did not touch the code. Zero lines. Full agent autonomy.
- Unlike most human users, Claude *reads the documentation*. One stuck agent? I said "Read the **grind** docs." It downloaded, understood, unblocked.
- Examples are powerful: agents found our DSL declaration examples and adopted the same pattern for Radix syntax.
- Claude struggled with *mutual induction* — but found workarounds every time.
- Claude struggled with *monadic code* and avoided it. We don't have enough examples for that.
- I created a GitHub token scoped to the Radix repo only. Good thing — Claude tried to push to **leanprover/lean4**.
- Toolchain version issues (v4.29.0) confused Claude when generating these slides.

Radix — A Real Program

Agents write Radix using concrete syntax macros. Verso elaborates every line:

```
def slideBubbleSort := `[RStmt|
  let arr := new uint64[][10];
  arr[0] := 5; arr[1] := 3; arr[2] := 8;
  arr[3] := 1; arr[4] := 9; arr[5] := 2;
  arr[6] := 7; arr[7] := 4; arr[8] := 6;
  arr[9] := 0;
  i := 0;
  while (i < 9) {
    j := 0;
    while (j < 9 - i) {
      let a : uint64 = arr[j];
      let b : uint64 = arr[j + 1];
      if (a > b) {
        arr[j] := b;
        arr[j + 1] := a;
      }
      j := j + 1;
    }
    i := i + 1;
  }
]
```


Big-Step Semantics — 16 Rules

```
-- Big-step:  $\langle \sigma, s \rangle \Downarrow r$ 
-- 16 rules: skip, assign, decl, seqNormal, seqReturn,
-- ifTrue, ifFalse, whileTrue, whileReturn, whileFalse,
-- alloc, free, arrSet, ret, block, callStmt, scope
example : BigStep  $\sigma$  .skip (.normal  $\sigma$ ) :=
  BigStep.skip
example (he : e.eval  $\sigma$  = some v)
  (hs :  $\sigma$ .setVar x v = some  $\sigma'$ ) :
  BigStep  $\sigma$  (.assign x e) (.normal  $\sigma'$ ) :=
  BigStep.assign he hs
example (h1 : BigStep  $\sigma_1$  s1 (.normal  $\sigma_2$ ))
  (h2 : BigStep  $\sigma_2$  s2 r) :
  BigStep  $\sigma_1$  (s1 ;; s2) r :=
  BigStep.seqNormal h1 h2
example (hc : e.eval  $\sigma_1$  = some (.bool true))
  (hb : BigStep  $\sigma_1$  b (.normal  $\sigma_2$ ))
  (hw : BigStep  $\sigma_2$  (.while e b) r) :
  BigStep  $\sigma_1$  (.while e b) r :=
  BigStep.whileTrue hc hb hw
```

Determinism — grind Does The Work

```
-- Same state + same statement → same result
-- 80-line proof by induction on  $h_1$ 
-- grind closes equational contradictions
example ( $h_1$  : BigStep  $\sigma$   $s$   $r_1$ )
  ( $h_2$  : BigStep  $\sigma$   $s$   $r_2$ ) :  $r_1 = r_2$  :=
  BigStep.det  $h_1$   $h_2$ 
```

In the real proof: `cases  $h_2$  with | assign => grind` handles equational cases. `grind` decides contradictions like `some true = some false`. Explicit IH application for recursive cases (`seq`, `while`, `call`, `scope`).

Interpreter Correctness — Sound and Complete

A fuel-based interpreter (`Stmt.interp`) proved equivalent to the relational semantics in both directions:

```
-- Completeness: BigStep implies interp succeeds
example (h : BigStep  $\sigma$  s r) :
   $\exists$  fuel, s.interp fuel  $\sigma$  =
    (.ok r.retVal, r.state) :=
    Stmt.interp_complete h
-- Soundness: interp success implies BigStep
example (h : s.interp fuel  $\sigma$  = (.ok rv,  $\sigma'$ )) :
  BigStep  $\sigma$  s (toStmtResult rv  $\sigma'$ ) :=
  Stmt.interp_sound h
```

The relational semantics is the spec. The interpreter is the implementation. These two theorems say they agree exactly.

Verified Optimizations — 5 Passes, 0 Sorry

```
-- Each pass: Stmt → Stmt with correctness theorem
-- "If the original runs, the optimized runs too"
example (h : BigStep σ s r) :
  BigStep σ s.constFold r :=
  Stmt.constFold_correct h
example (h : BigStep σ s r) :
  BigStep σ s.deadCodeElim r :=
  Stmt.deadCodeElim_correct h
example (h : BigStep σ s r) :
  BigStep σ s.copyPropagation r :=
  Stmt.copyProp_correct h
example (h : BigStep σ s r) :
  BigStep σ s.constPropagation r :=
  Stmt.constPropagation_correct h
example (h : BigStep σ s r) (hf : σ.funs = funs) :
  ∀ depth, BigStep σ (s.inline funs depth) r :=
  Stmt.inline_correct h hf
```


Optimizations — What The Proofs Actually Look Like

Constant folding: `Expr.inferTag` conservatively infers output types to guard identity rules. `$e + 0 \rightarrow e$` is sound when `e` produces `uint64`. `$e * 0 \rightarrow 0$` is NOT sound — if `e` fails (e.g., out-of-bounds), the original returns `none` but the rewrite returns `some 0`.

Copy propagation: `CopyMap.agrees` invariant — every mapping `$x \rightarrow y$` means `$\sigma.\text{getVar } x = \sigma.\text{getVar } y$` . Reset at control flow joins to stay sound.

Inline: rewrites `callStmt` into `scope` (frame-isolated body), bounded depth, non-recursive only. Proves function table invariant preserved through execution.

Linear Ownership — Soundness

```
-- LinearOk 0 s 0' : owned-set 0 → stmt s → owned-set 0'
-- 13 rules. Key ones:
example : LinearOk 0 .skip 0 :=
  LinearOk.skip
example (h : x ∉ 0) :
  LinearOk 0 (.alloc x ty sz) (0.insert x) :=
  LinearOk.alloc h
example (h : x ∈ 0) :
  LinearOk 0 (.free (.var x)) (0.erase x) :=
  LinearOk.free h
example (h1 : LinearOk 0 s1 0')
  (h2 : LinearOk 0' s2 0'') :
  LinearOk 0 (s1 ;; s2) 0'' :=
  LinearOk.seq h1 h2
```

Linear Ownership — The Real Invariant

The soundness proof (644 lines) maintains `OwnershipInv` — a three-part invariant:

- **(1) Heap well-formed:** all stored addresses < `nextAddr`
- **(2) Liveness:** every owned variable holds a live heap address
- **(3) Distinctness:** different owned variables hold different addresses

```
-- Soundness: OwnershipInv preserved through execution
example (hlin : LinearOk 0 s 0')
  (hstep : BigStep  $\sigma$  s (.normal  $\sigma'$ ))
  (hinv : OwnershipInv  $\sigma$  0)
  (hwt : WellTypedFuns  $\sigma$ .funs) :
  OwnershipInv  $\sigma'$  0' :=
  LinearOk.soundness hlin hstep hinv hwt
```

Proved by induction on BigStep — every rule preserves all three parts. The `alloc` case uses `Heap.alloc_fresh` for distinctness. The `free` case uses `Heap.free_preserves_ne`.

The 10 Agents

Not 10 interchangeable "coding agents." 10 *domain experts*:

- *Chris Lattner* — AST, IR, module structure
- *Simon Peyton Jones* — types, syntax macros
- *Xavier Leroy* — verification strategy
- *Adam Chlipala* — proof engineering
- *Emina Torlak* — type checking, tests
- *Derek Dreyer* — semantics, memory model
- *John Regehr* — edge cases, coverage
- *Dan Grossman* — memory safety proofs
- *Nadia Polikarpova* — formal specifications
- *Tiark Rompf* — interpreter, optimizations

All Claude. One weekend.

The Surprise

The agents gave us *better feedback on Lean* than our human users.

- More precise: structured error context, not "it doesn't work"
- More systematic: they hit every edge case, not just the one they needed
- More actionable: "tactic X fails on pattern Y because Z"

Adam's **grind** report was better than most human post-mortems.

Optimize Lean for AI too, not just humans.

Nobody has built this yet.

The Spec and The Feedback Loop

- *Better diagnostics*: when a tactic fails, structured error metadata — not just prose. Claude happily processes them.
- *Links to documentation*: tactic docstrings and error messages should contain links to the actual documentation. Claude reads them.
- *Faster startup*: Claude can try different approaches more efficiently.
- *More examples*: and instructions on how to find them. Claude started the Radix project by reading all our examples in the core repo.
- *Attribute guides*: teach Claude how to use `@[simp]` , `@[grind]` , etc.
- *Implicit information*: a pretty printer that shows implicit arguments and avoids exponential blowup. Claude claims it will help — confirming with Anthropic.

Next experiment: Two agent teams in parallel. One builds a hard project and generates friction reports. The other patches Lean to eliminate the friction. We architect and steer.

A Probable Future

- People will not write Lean. AI will write it. Humans read and audit.
- People will not write Verso. AI will produce lectures, papers, slides — with machine-checked math and code. Humans focus on the content.

Humans move up the stack. They become designers, architects, readers, auditors.